LETI: Learned Space Filling Curves for Efficient Trajectory Indexing in Key-Value Databases

Tianyu Huang
College of Computer Science
Chongqing University
Chongqing, China

Yingqi Tang
College of Computer Science
Chongqing University
Chongqing, China

Ruiyuan Li
College of Computer Science
Chongqing University
Chongqing, China

***Abstract*—Large-scale trajectory data are increasingly managed in key-value (KV) stores, where query processing is dominated by range scans over keys followed by refinement. In this setting, the design of trajectory keys directly determines query efficiency. Existing methods rely on fixed spatial hierarchies, predefined or globally learned space-filling curves, and fixed shape partitions, which often lead to fragmented scan ranges and excessive candidates, especially under skewed and heterogeneous trajectory distributions. We propose LETI, a trajectory indexing framework that jointly optimizes spatial hierarchy, traversal order, and shape representation. LETI constructs a compact quadtree by pruning low-density regions, learns a traversal order over retained cells using reinforcement learning, and adapts shape partitions within each enlarged element according to local trajectory distributions. This joint design produces more compact scan ranges and reduces the number of candidates passed to refinement. Experiments on real-world datasets show that LETI consistently improves query efficiency, reducing latency and improving filtering effectiveness across diverse workloads. Overall, LETI improves the query performance of the best-performing baseline by up to 32.4%.**

***Index Terms*—Trajectory Indexing, Key-Value Databases, Learned Index, Space-Filling Curves, Reinforcement Learning**

I. Introduction

Trajectories are a fundamental form of spatio-temporal data that record object movement as sequences of time-ordered locations [1]. With the proliferation of mobile devices, location-based services, and Internet of Things (IoT) systems, large-scale trajectory collections have become a key data foundation for a wide range of applications, including trajectory pattern mining [2], path prediction [3], and point-of-interest (POI) discovery [4]. As both the volume and importance of trajectory data continue to grow, efficient indexing and query processing have become a central system problem. Key-value (KV) stores are well suited for this setting due to their high write throughput, low access latency, and horizontal scalability [5]. In KV-based trajectory management, trajectories (or trajectory segments) are organized by keys, while their detailed spatio-temporal contents are stored as values. As a result, query processing is largely reduced to range scans over keys followed by refinement, making the design of trajectory keys a primary factor in overall query efficiency.

Designing such keys is challenging because trajectories are spatially extended objects rather than points. An effective index must map spatial information into a one-dimensional key space while preserving locality and retaining enough shape information for pruning. Existing approaches address this problem from two directions. Spatial encoding methods use a hierarchy together with space-filling curves (SFCs) to linearize trajectory locations. As illustrated in Fig. 1a, XZ2 [6] maps each trajectory to the cell whose enlarged element (EE) covers its minimum bounding rectangle (MBR), so that spatial queries can be translated into contiguous key intervals for efficient scans. Subsequent works [7]–[10] augment this design with shape encoding by partitioning the EE into subcells and representing trajectory coverage as bitmaps, where each bit indicates whether the trajectory intersects the corresponding subcell. This allows incompatible trajectories to be filtered early during scans, reducing false positives. The latter seeks better linearization orders by making SFC construction data-aware or workload-aware. Despite these advances, existing trajectory indexes still suffer from three fundamental limitations.

Challenge 1: Spatial Hierarchy Mismatch. Fixed spatial hierarchies often preserve many low-density cells even when they contain few or no trajectories. As shown in Fig. 1b, These cells are still encoded into the key space and may separate dense cells such as A, B, and C, introducing gaps that weaken locality and enlarge the search space during query processing.

Challenge 2: Traversal Order Mismatch. Even when relevant cells are identified, their ordering in the key space may fail to preserve spatial and distributional locality. Fig. 1b shows that query-relevant cells can be separated by unrelated ones, causing fragmented scans. Ideally, a better traversal places spatially nearby and frequently co-accessed cells together. However, existing predefined or globally learned orders are often insensitive to local trajectory distributions and workload-induced access patterns, leading to fragmented key intervals, unnecessary accesses, and higher refinement overhead.

Challenge 3: Shape Mismatch. Trajectory distributions vary significantly across regions, making a single fixed EE partition insufficient. As illustrated in Fig. 1c, a fine partition causes nearby trajectories such as T_1 and T_2 to generate redundant encodings, while a coarse partition makes trajectories such as T_4 and T_5 insufficiently distinguishable. This mismatch leads to excessive encoding overhead or weak filtering.

These issues are closely related. A mismatched hierarchy introduces structural gaps that limit how well a traversal can

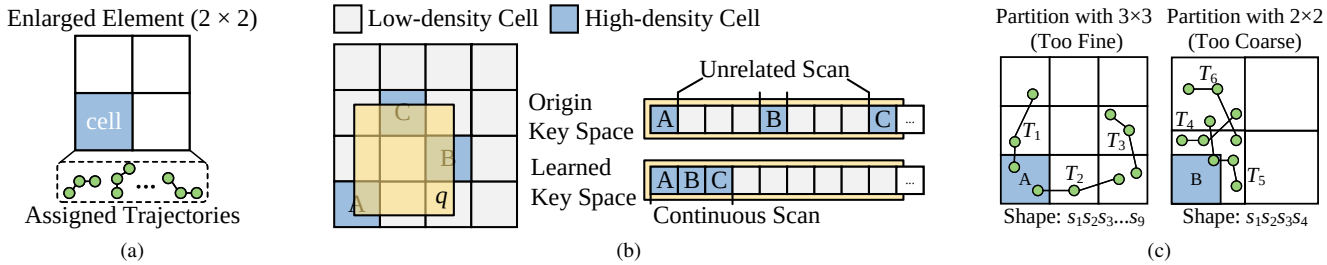


Fig. 1. Limitations of existing trajectory key designs. (a) A trajectory is assigned to the cell whose EE covers its MBR. (b) Low-density cells and unrelated cells may be interleaved in key order, causing fragmented scans. (c) Fixed EE partitions may be too fine or too coarse, which either introduce redundant encoding or weaken filtering effectiveness.

preserve locality, while rigid shape encoding weakens pruning even when locality improves. As a result, focusing on any single component alone does not suffice for efficient query processing in KV stores. To address these issues, we propose **LETI** (**L**earned Space-Filling Curves for **E**fficient **T**rajectory **I**ndexing), a trajectory indexing framework that refines both spatial encoding and shape encoding for KV stores. LETI first constructs a compact quadtree [11] by pruning low-density cells, so that the retained hierarchy better matches the trajectory distribution and avoids structural gaps in the key space. It then learns a traversal order over the retained cells on the compact quadtree, producing a data- and workload-aware space-filling curve that reduces fragmented key intervals during query localization. Finally, LETI adaptively selects the shape partition within each enlarged element according to the local trajectory distribution, improving the discriminability of shape encodings for filtering. In summary, we make the following contributions:

- We propose LETI, a trajectory indexing framework for KV stores that jointly optimizes spatial hierarchy, traversal order, and shape encoding.
- We formulate SFC construction as a sequential decision problem and develop an RL-based method that captures both workload patterns and trajectory distribution.
- We design an adaptive shape encoding scheme that selects partitions according to local trajectory distributions to improve the discriminability of shape-based filtering.
- We conduct extensive experiments on two real-world datasets, showing that LETI consistently outperforms baselines.

Paper Organization. Section II introduces the preliminaries, followed by an overview of LETI in Section III. Section IV presents the proposed method in detail, and Section VI reports the experimental results. Related work is discussed in Section VII, and Section VIII concludes the paper. For ease of reference, Table I summarizes the main notations.

II. Preliminaries

A. Definitions

Definition 1 (Trajectory). A trajectory $\mathcal{T}$ is an ordered sequence of spatio-temporal points:

$$\mathcal{T} = \{p_1, p_2, \ldots, p_m\}, \quad p_i = (x_i, y_i, t_i), \tag{1}$$

TABLE I
Table of notations.

Symbol	Meaning
$\mathbf{T}$ (resp. $\mathcal{T}$)	the trajectory dataset (resp. a trajectory)
q (resp. $\mathcal{W}$)	a spatial query window (resp. a query workload set)
h (resp. ℓ)	the maximum quadtree depth (resp. a quadtree level)
$\mathcal{T}(c)$	the set of trajectories assigned to cell c
$\mathrm{MBR}(\mathcal{T})$	the minimum bounding rectangle of trajectory $\mathcal{T}$
$\mathrm{EE}(c)$	the enlarged element of cell c
$\pi(c)$	the one-dimensional spatial encoding assigned to cell c
$n_x \times n_y$	the partition size of the enlarged element
$s(\mathcal{T})$ (resp. $s^*(\mathcal{T})$)	the shape bitmap (resp. compact local shape code) of trajectory $\mathcal{T}$

where (x_i, y_i) denotes spatial coordinates and t_i is the timestamp, with $t_1 < t_2 < \cdots < t_m$.

Definition 2 (Query Types). Three trajectory query types are considered: spatial range query, similarity search, and top-k query. Let $q = (x_{\min}, y_{\min}, x_{\max}, y_{\max})$ denote a spatial query window, $\mathbf{T} = \{\mathcal{T}_1, \ldots, \mathcal{T}_n\}$ a trajectory dataset, $\mathcal{T}_q$ a query trajectory, τ a similarity threshold, and $d(\cdot,\cdot)$ a trajectory distance function (e.g., Fréchet [12] or Dynamic Time Warping (DTW) [13]). The queries are defined as follows.

- **Spatial Range Query (SRQ).** Given a spatial query window q, an SRQ returns all trajectories intersecting q, i.e.,

$$\mathcal{T}_{\mathrm{srq}} = \{\mathcal{T}_i \in \mathbf{T} \mid \mathcal{T}_i \cap q \neq \emptyset\}. \tag{2}$$

- **Similarity Search (SS).** Given a query trajectory $\mathcal{T}_q$ and a similarity threshold τ, an SS returns all trajectories whose distance to $\mathcal{T}_q$ does not exceed τ. Formally,

$$\mathcal{T}_{\mathrm{ss}} = \{\mathcal{T}_i \in \mathbf{T} \mid d(\mathcal{T}_q, \mathcal{T}_i) \leq \tau\}. \tag{3}$$

- **Top-k Query.** Given a query trajectory $\mathcal{T}_q$ and an integer k, a top-k query returns the k trajectories in $\mathbf{T}$ with the smallest distances to $\mathcal{T}_q$. Formally,

$$\text{top-}k(\mathbf{T}, \mathcal{T}_q, k) = \arg\min_{\mathcal{S} \subseteq \mathbf{T}, |\mathcal{S}|=k} \sum_{\mathcal{T}_i \in \mathcal{S}} d(\mathcal{T}_q, \mathcal{T}_i). \tag{4}$$

Both similarity search and top-k queries can be evaluated within the same range-localization pipeline as SRQ by transforming distance constraints into spatial windows (e.g., via bounding regions and refinement), as in TrajMesa [7]. We focus on SRQ for index design and query processing, while reporting results for SS and top-k queries in Section VI.

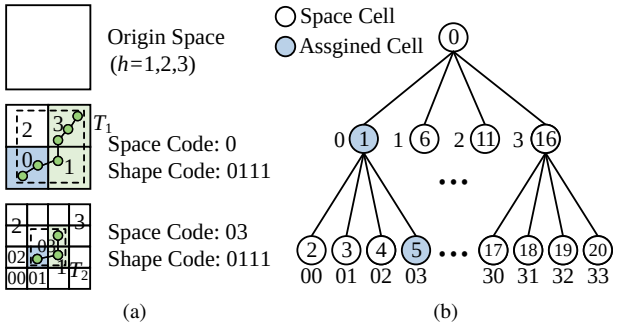


Fig. 2. Trajectory representation with spatial and shape encodings. (a) T_1 and T_2 are assigned to cells 0 and 03, respectively, yet share the same shape bitmap (0111) under a 2×2 enlarged-element partition. (b) Spatial cells form a quadtree, where node labels denote paths and integers denote XZ-order encodings; highlighted nodes correspond to the assigned cells.

Definition 3 (Space-Filling Curve, SFC). A space-filling curve is a mapping

$$\pi : \mathbb{R}^D \to \mathbb{R} \tag{5}$$

that transforms multi-dimensional spatial locations into a one-dimensional space while preserving spatial locality.

Definition 4 (Minimum Bounding Rectangle, MBR). The minimum bounding rectangle of a trajectory $\mathcal{T}$, denoted as $\mathrm{MBR}(\mathcal{T})$, is the smallest axis-aligned rectangle that encloses all points in $\mathcal{T}$:

$$\mathrm{MBR}(\mathcal{T}) = (x_{\min}, y_{\min}, x_{\max}, y_{\max}). \tag{6}$$

Definition 5 (Enlarged Element, EE). Given a spatial region $R = (x_{\min}, y_{\min}, x_{\max}, y_{\max})$, let $w_R = x_{\max} - x_{\min}$ and $h_R = y_{\max} - y_{\min}$ denote its width and height. The enlarged element of R is defined by doubling its width and height:

$$\mathrm{EE}(R) = (x_{\min}, y_{\min}, x_{\min} + 2w_R, y_{\min} + 2h_R). \tag{7}$$

B. Trajectory Encoding

To support indexing in KV stores, each trajectory is mapped to a one-dimensional key through a two-part representation consisting of a *spatial encoding* and a *shape encoding*, as illustrated in Fig. 2.

Hierarchical Spatial Partitioning. The spatial domain is recursively partitioned by a quadtree up to a maximum depth h. Each cell corresponds to a node in the hierarchy and is associated with (i) a quadtree path label indicating its position in the tree, and (ii) a one-dimensional spatial encoding $\pi(c)$ induced by the XZ-order. As shown in Fig. 2b, cell 0 has descendants 00, 01, 02, and 03, whose corresponding encodings are given by the integers inside the nodes.

Spatial Encoding Assignment. Given a trajectory $\mathcal{T}$, let $\mathrm{MBR}(\mathcal{T})$ denote its minimum bounding rectangle. Its assigned cell is determined in two steps.

1) *Level Selection.* Choose the finest level ℓ such that there exists a cell c at level ℓ whose enlarged element $\mathrm{EE}(c)$ fully covers $\mathrm{MBR}(\mathcal{T})$.
2) *Cell Assignment.* Among the cells at level ℓ, select the cell containing the lower-left corner $(x_{\min}, y_{\min})$ of $\mathrm{MBR}(\mathcal{T})$. The assigned cell determines both the quadtree path label and its one-dimensional spatial encoding $\pi(c)$. We denote the assigned cell of a trajectory $\mathcal{T}$ as $c_{\mathcal{T}}$, and $\pi(c_{\mathcal{T}})$ as its one-dimensional integer encoding. In Fig. 2a, trajectory T_1 is assigned to cell 0 at level 1, whereas T_2 to cell 03 at level 2.

Shape Encoding. To encode local coverage inside $\mathrm{EE}(c)$, the enlarged element is partitioned into an $n_x \times n_y$ grid, and the trajectory is represented as a bitmap $s(\mathcal{T}) \in \{0,1\}^{n_x \times n_y}$, where each bit indicates whether $\mathcal{T}$ intersects the corresponding subcell under a fixed bit order. In Fig. 2a, the shadow subcells are those intersected by the trajectory and are therefore set to 1; both T_1 and T_2 yield the same shape bitmap 0111, although their quadtree path labels are different.

A trajectory key combines the spatial encoding $\pi(c)$ and the shape encoding. Because the shape space under an $n_x \times n_y$ partition contains up to $2^{n_x \times n_y}$ bitmaps, directly encoding all shapes is inefficient. Following TShape [10], raw shape bitmaps are compacted locally within each cell. Let $\mathcal{S}(c) = \{s(\mathcal{T}) \mid \mathcal{T} \in \mathcal{T}(c)\}$ denote the set of distinct shapes in cell c. Each shape bitmap is remapped to a compact local shape code $s^*(\mathcal{T}) \in \{1, 2, \ldots, |\mathcal{S}(c)|\}$. The index value is

$$\mathrm{LETI}(\mathcal{T}) = (\pi(c_{\mathcal{T}}) \ll b) \mid s^*(\mathcal{T}), \tag{8}$$

where $\ll$ denotes left shift and b is the number of bits reserved for compact shape codes. Let $N = \max_{c \in \mathcal{C}} |\mathcal{S}(c)|$ be the maximum number of distinct shapes across all cells. and set $b = \lfloor \log_2 N \rfloor + 1$ so that compact shape codes from different cells do not overlap.

$$\mathrm{key}(\mathcal{T}) = \mathrm{shard} :: \mathrm{LETI}(\mathcal{T}) :: \mathrm{tid}, \tag{9}$$

where shard is used for load balancing and tid uniquely identifies the trajectory. Following Trass [9], we use 8 shards, i.e., $\mathrm{shard} \in \{0, 1, \ldots, 7\}$. Our focus is on $\mathrm{LETI}(\mathcal{T})$, while the shard prefix and tid follow standard KV-store conventions.

C. Retrieval Workflow

Given a spatial range query q, query processing follows a filter-and-refine workflow that narrows the search space using both spatial and shape encodings. Let $\mathcal{C}_q$ denote the set of cells whose enlarged elements intersect q.

1) *Spatial Filtering.* Identify all cells $c \in \mathcal{C}_q$ such that $\mathrm{EE}(c) \cap q \neq \varnothing$, and use their spatial encodings $\pi(c)$ to locate corresponding key regions. Each $\pi(c)$ is associated with a shape cache $\mathrm{ShapeCache}(c)$, which stores distinct shape tuples $\langle s, s^* \rangle$ in cell c, where s is the raw shape bitmap and s^* its compact local code.
2) *Shape Filtering.* For each candidate cell c, construct a query shape signature $s_q(c)$ over $\mathrm{EE}(c)$ using the same $n_x \times n_y$ partition as trajectory encoding. A bit is set to 1 if the corresponding subcell intersects q. The signature is matched against $\mathrm{ShapeCache}(c)$, and a tuple $\langle s, s^* \rangle$ is retained only if $s_q(c) \wedge s \neq 0$.
3) *Key Scan and Refinement.* The retained local shape codes are combined with $\pi(c)$ to form candidate key prefixes (i.e., intervals containing a single key). These prefixes are

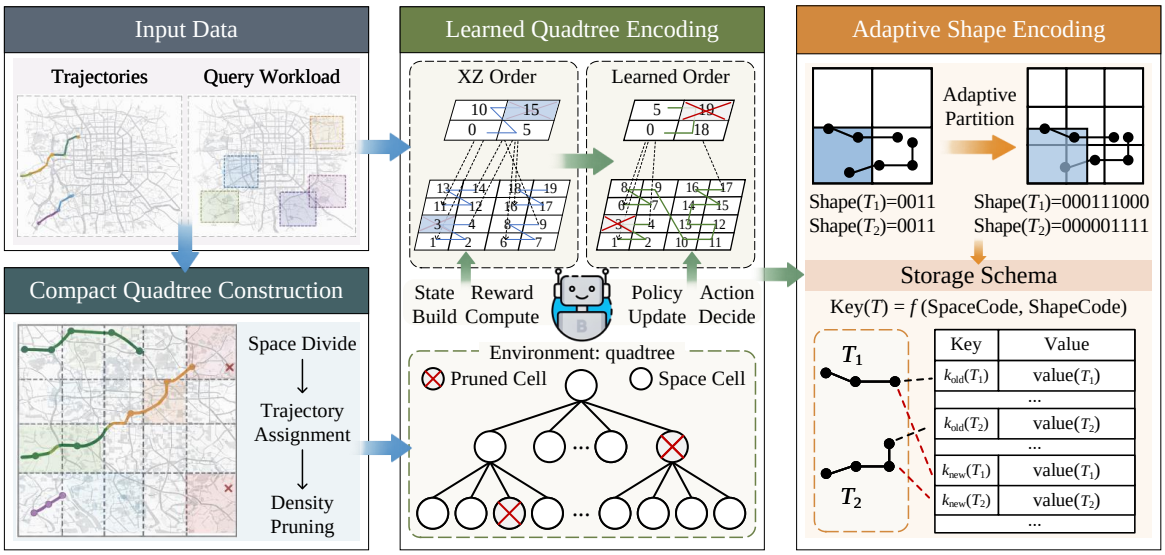


Fig. 3. Overview of the LETI framework.

scanned in the KV store to retrieve candidate trajectories, which are then verified by exact geometric intersection.

Limitations. The efficiency of this workflow depends on producing compact key intervals and effective filtering. If spatially related cells are scattered in the key order, a query may generate many fragmented key intervals, increasing scan overhead. Similarly, if trajectories share the same shape encoding, filtering becomes less selective and more candidates pass to refinement. Both issues increase scan and verification cost, motivating the design of LETI.

III. Overview of LETI

The overall framework of LETI is illustrated in Fig. 3. LETI improves trajectory indexing by refining both spatial encoding and shape encoding. By constructing a more compact spatial structure together with a more effective shape encoding, LETI produces trajectory keys that reduce interval localization and scan overhead during query processing in KV stores. The framework consists of three stages.

(1) Compact Quadtree Construction. This stage refines the spatial representation. A quadtree is constructed according to the trajectory distribution, and cells with low trajectory density are removed. The subsequent encoding is then performed on the resulting compact structure.

(2) Learned Quadtree Encoding. This stage determines the traversal order over the compact quadtree. Instead of using a predefined spatial order, LETI adopts a RL-based approach to learn a space-filling curve. The learned traversal order better matches trajectory and query distributions and helps preserve spatial locality in the key space.

(3) Adaptive Shape Encoding. This stage refines the shape encoding within each enlarged element. LETI selects the partition adaptively according to the local trajectory distribution. By balancing space utilization and shape discriminability, it improves the effectiveness of shape-based filtering.

Algorithm 1 Bottom-up Density-based Quadtree Pruning

Input: Quadtree $\mathcal{Q}$, threshold δ
Output: Pruned quadtree $\mathcal{Q}'$

```
for each level ℓ from bottom to top do
    for each cell c at level ℓ do
        if |T(c)| < δ and c is not root then
            p ← parent(c)
            T(p) ← T(p) ∪ T(c)
            reassign T(c) to p
            remove c from Q
return Q'
```

IV. Methodology

A. Compact Quadtree Construction

The original spatial partitioning scheme recursively subdivides space into hierarchical cells, forming a quadtree that is subsequently linearized into one-dimensional codes. However, this design is not well suited for trajectory indexing due to its uneven structure. In particular, spatial proximity is not consistently preserved in the key space. Even nearby trajectories may be mapped to distant codes when their corresponding cells are separated by many intermediate nodes in the traversal order.

This issue is more pronounced on real-world datasets with skewed trajectory distributions. As shown in Fig. 4, trajectories concentrate in a few levels (e.g., levels 5–6). In deeper level, most cells are assigned with only a handful of trajectories (fewer than 5). These low-density cells still introduce long gaps in the encoding, separating dense regions in the key space and weakening locality.

To address this issue, LETI constructs a compact quadtree by pruning nodes with few assigned trajectories. Formally, each trajectory $\mathcal{T}$ is assigned to a space cell c according to its spatial extent $\mathrm{MBR}(\mathcal{T})$, and $\mathcal{T}(c)$ denotes the set of trajectories associated with c. Given a pruning threshold δ, any node c with $|\mathcal{T}(c)| < \delta$ is removed, and its trajectories

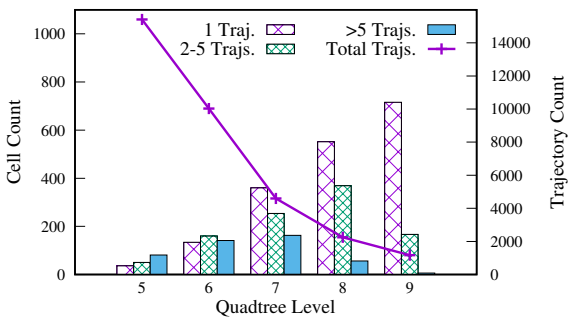


Fig. 4. Trajectory distribution across quadtree levels. Trajectory assignments are concentrated in a few middle levels, while deeper levels contain a high fraction of sparse cells.

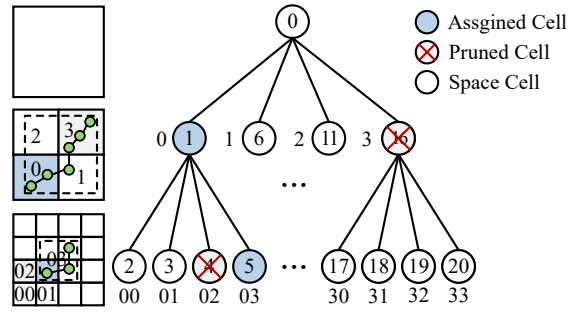


Fig. 5. Density-based quadtree pruning. Cells with fewer than δ trajectories (e.g., 4 and 16) are removed, and their trajectories are reassigned to ancestor cells (e.g., 1 and 0).

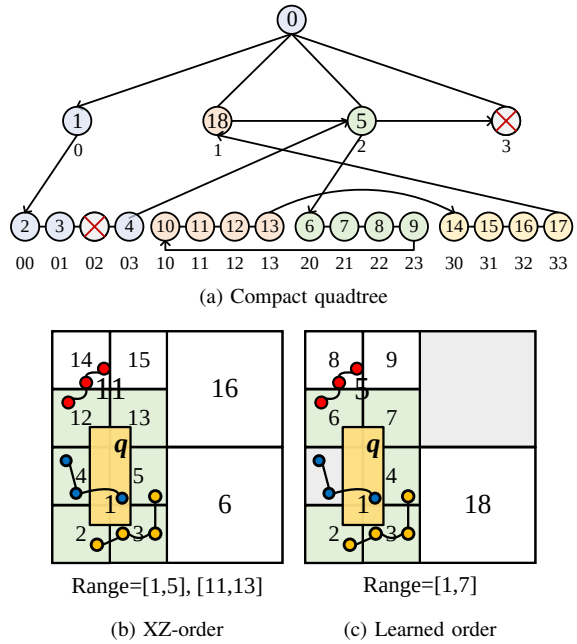


Fig. 6. Effect of traversal order on key-interval generation. (a) A learned traversal over the pruned quadtree, where low-density nodes are removed. (b) Under XZ-order, relevant cells are separated, producing fragmented intervals (e.g., $[1, 5]$, $[11, 13]$). (c) The learned traversal places them contiguously, merging into a single interval $[1, 7]$.

are propagated to the nearest ancestor node that satisfies the threshold. The pruning is performed bottom-up using post-order traversal. Starting from the leaves, each node is examined and removed if it does not meet the threshold, while its trajectories are reassigned to its parent. As illustrated in Fig. 5, nodes such as 4 and 16 are removed, and their trajectories are reassigned to nodes such as 1 and 0.

Algorithm 1 summarizes the procedure. It takes a quadtree $\mathcal{Q}$ and a density threshold δ as input, and outputs a pruned quadtree $\mathcal{Q}'$. The tree is scanned bottom-up from leaves to the root (Lines 3–4). For each cell c, the algorithm checks whether $|\mathcal{T}(c)| < \delta$ (Line 5). If so, it identifies the parent cell p (Line 6), propagates $\mathcal{T}(c)$ to p (Lines 7–8), and removes c (Line 9). After all levels are processed, the pruned hierarchy is returned.

Let N denote the number of quadtree cells and M the total number of trajectory assignments. Each cell is visited once, yielding $O(N)$ time, while trajectory reassignment incurs $O(M)$ total cost. The overall time complexity is $O(N + M)$, reducing to $O(N)$ when only cell densities are maintained incrementally. The space overhead is $O(N)$.

B. Learned Quadtree Encoding

While the compact quadtree removes structural redundancy, it does not determine how retained cells are mapped into one-dimensional keys. This linearization step is critical in KV stores, as the traversal order directly affects the number and compactness of key intervals produced during range localization. If related cells are placed far apart in the key space, a single spatial query may result in multiple fragmented scans, increasing both index lookup and data access cost.

Existing spatial encoding schemes are limited for this purpose. Fixed space-filling curves such as XZ-order are independent of both trajectory distribution and query workload, and do not adapt to the uneven density patterns observed in real data. Workload-driven approaches, in contrast, tend to fit a particular query distribution and may degrade when the workload changes. More fundamentally, neither predefined nor workload-specific orders explicitly preserve locality with respect to both spatial proximity and trajectory distribution.

For trajectory indexing, the traversal order should reflect both *geometric proximity* and *distributional affinity*. Cells with similar trajectories are often co-accessed and should be placed nearby in the key space. The compact quadtree exposes both spatial structure and trajectory distribution, providing a natural basis for constructing such a traversal. To reduce interval fragmentation during range localization, LETI learns a traversal order directly over the compact quadtree. As illustrated in Fig. 6(a), the traversal is defined after low-density nodes are pruned, where crossed-out nodes do not participate in the ordering. Restricting the traversal to retained nodes places spatially close cells with similar trajectory distributions near each other in the traversal sequence.

Without this adaptation, fixed traversal orders may separate query-relevant cells in the key space. As shown in Fig. 6(b), under XZ-order, cells intersecting the same query window q are mapped to disjoint positions. Although these cells are spatially related, their separation in the key space translates the query into multiple disjoint intervals, such as $[1, 5]$ and

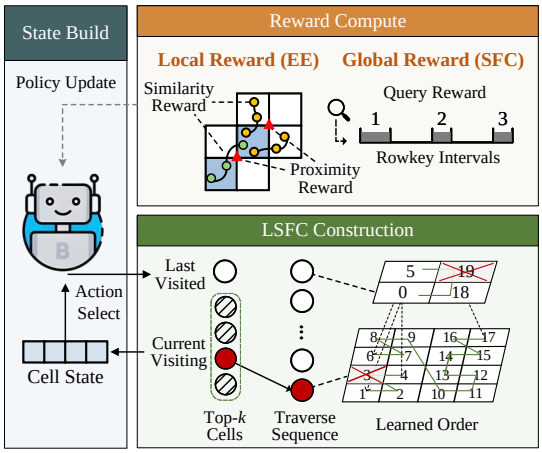


Fig. 7. RL framework for learned space-filling curve construction.

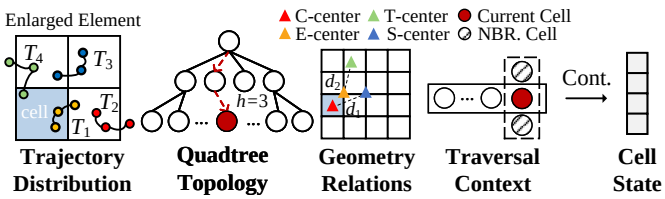


Fig. 8. State representation of a cell.

[11, 13], leading to fragmented scans and increased access overhead. The learned traversal in Fig. 6(c) places these cells contiguously in the key space. As a result, the same query window q can be covered by a single interval $[1, 7]$. This reduction in fragmented intervals lowers the number of generated scan ranges and improves scan locality.

Problem Formulation. Let $\mathcal{C} = \{c_1, \ldots, c_N\}$ denote the set of cells in the pruned quadtree, and let $\mathcal{W} = \{q_1, \ldots, q_K\}$ denote a query workload. Each cell c_i is associated with a set of trajectories $\mathcal{T}(c_i)$. We aim to learn a bijective mapping

$$\pi : \mathcal{C} \to \{1, 2, \ldots, |\mathcal{C}|\}, \tag{10}$$

which defines a traversal order over the cells. The objective is to place cells that are closer in space and more similar in trajectory distribution at nearby positions in the one-dimensional key space, so that queries from $\mathcal{W}$ can be evaluated with fewer and more compact key intervals.

RL Formulation. Finding an optimal mapping π is a combinatorial optimization problem with a search space of size $|\mathcal{C}| \times (|\mathcal{C}| - 1) \ldots \times 1$, which quickly becomes intractable as $|\mathcal{C}|$ grows. We therefore model the construction of the traversal order as a Markov Decision Process (MDP) and learn a data-aware space-filling curve using reinforcement learning, as illustrated in Fig. 7. Specifically, the traversal is constructed incrementally. At step t, the agent selects a cell from the remaining candidates and appends it to the current sequence. Each decision affects both local continuity and the structure of the final traversal order, and is evaluated during training.

State Representation. At decision step t, a candidate cell c_t is represented by $s_t = \phi(c_t, \mathcal{T}(c_t), \pi_t)$, where $\mathcal{T}(c_t)$ denotes the trajectories associated with c_t, and π_t is the partial traversal constructed so far. As illustrated in Fig. 8, the state combines four types of features.

1) *Trajectory Distribution.* The first component characterizes trajectories around c_t based on their spatial relations with the cell and its enlarged element $\mathrm{EE}(c_t)$. As shown in Fig. 8, trajectories are grouped into four categories: (i) intersecting c_t and covered by $\mathrm{EE}(c_t)$ (e.g., T_1), (ii) intersecting $\mathrm{EE}(c_t)$ but not c_t (e.g., T_2), (iii) covered by $\mathrm{EE}(c_t)$ but not intersecting c_t (e.g., T_3), and (iv) intersecting both c_t and $\mathrm{EE}(c_t)$ (e.g., T_4). Their normalized proportions form $\mathbf{f}_{\mathrm{traj}}(c_t)$, capturing local density and distribution patterns.
2) *Quadtree Topology.* The second component encodes the position of c_t in the compact quadtree, including its level and path from the root. These features form $\mathbf{f}_{\mathrm{quad}}(c_t)$, providing hierarchical context beyond spatial coordinates.
3) *Geometric Relations.* The third component captures relative spatial relations using representative centers, including the cell center (C-center), EE center (E-center), trajectory centroid (T-center), and global centroid (S-center). Pairwise distances among these centers, such as d_1 and d_2 in Fig. 8, form $\mathbf{f}_{\mathrm{geo}}(c_t)$, capturing geometric proximity and distributional affinity.
4) *Traversal Context.* The final component captures dynamic information from the current traversal, including the visited ratio $|\pi_t|/|\mathcal{C}|$, the distance to the previously selected cell, and density statistics of neighboring cells. These features form $\mathbf{f}_{\mathrm{ctx}}(c_t)$, guiding the next transition.

The final state representation concatenates all components:

$$s_t = [\mathbf{f}_{\mathrm{traj}}(c_t), \mathbf{f}_{\mathrm{quad}}(c_t), \mathbf{f}_{\mathrm{geo}}(c_t), \mathbf{f}_{\mathrm{ctx}}(c_t)]\,. \tag{11}$$

Action Space and Transition. At step t, let c_{t-1} denote the cell selected at the previous step, and let $\mathcal{V}_{t-1} \subseteq \mathcal{C}$ denote the set of visited cells. The action is to select the next cell c_t from the remaining set $\mathcal{C} \setminus \mathcal{V}_{t-1}$. A naive formulation that allows transitions to any unvisited cell leads to an action space of size $O(|\mathcal{C}|)$ at each step, which makes it difficult to preserve spatial locality during exploration. To reduce the search space and enforce locality, LETI restricts each decision to a local candidate set centered at c_{t-1}. Formally,

$$\mathcal{A}(c_{t-1}) = \mathcal{N}_{\text{same}}(c_{t-1}) \cup \mathcal{N}_{\text{adj}}(c_{t-1}), \tag{12}$$

where $\mathcal{N}_{\text{same}}(c_{t-1})$ denotes neighbors of c_{t-1} at the same quadtree level, and $\mathcal{N}_{\text{adj}}(c_{t-1})$ denotes adjacent-level neighbors, including its parent and child nodes. The valid action set at step t is $\widetilde{\mathcal{A}}_t = \mathcal{A}(c_{t-1}) \cap (\mathcal{C} \setminus \mathcal{V}_{t-1})$. If all cells in the local candidate set have already been visited, i.e., $\widetilde{\mathcal{A}}_t = \varnothing$, the transition falls back to the nearest unvisited cell in $\mathcal{C} \setminus \mathcal{V}_{t-1}$ according to spatial distance. The policy therefore considers only cells that are both local neighbors of c_{t-1} and unvisited, which reduces exploration complexity and encourages spatially coherent transitions.

Given the current state s_t, the policy network outputs a probability distribution over $\widetilde{\mathcal{A}}_t$. To further limit exploration, LETI retains only the top-k cells with the highest probabilities and samples c_t from the truncated distribution. After selecting

c_t, the visited set is updated as $\mathcal{V}_t = \mathcal{V}_{t-1} \cup \{c_t\}$, and the next state s_{t+1} is constructed from the updated partial traversal.

Reward Design. The reward is designed to capture two aspects of a high-quality traversal: (i) consecutive cells should exhibit strong local coherence, and (ii) the final traversal order should reduce the query cost induced by the workload. Accordingly, LETI adopts a two-level reward consisting of local and global components.

Local Reward. In early stages of sequence construction, the global quality of the traversal is not yet observable. We therefore provide dense local feedback to guide the policy toward transitions that preserve spatial and distributional consistency.

Given consecutive cells c_t and c_{t+1}, we define a proximity reward based on their normalized center distance d_{ctr}:

$$\text{Rew}_{\text{prox}}(c_t, c_{t+1}) = 1 - \frac{d_{\text{ctr}}(c_t, c_{t+1})}{d_{\text{ctr}}^{\max}}. \tag{13}$$

To capture the likelihood that two cells co-occur in query results, we further define a similarity reward based on trajectory overlap, corresponding to the Jaccard similarity [14] between their trajectory sets:

$$\text{Rew}_{\text{sim}}(c_t, c_{t+1}) = \frac{|\mathcal{T}(c_t) \cap \mathcal{T}(c_{t+1})|}{|\mathcal{T}(c_t) \cup \mathcal{T}(c_{t+1})|}. \tag{14}$$

Treating spatial proximity and trajectory similarity equally, the local reward is defined as

$$\text{Rew}_{\text{local}} = (\text{Rew}_{\text{prox}} + \text{Rew}_{\text{sim}})/2, \tag{15}$$

Global Reward. Local coherence alone does not ensure a globally effective traversal. We therefore evaluate the current traversal using a global reward derived from the estimated query behavior on workload $\mathcal{W}$. At step t, the learned policy only determines a partial traversal over visited cells. To evaluate its global effect, the remaining unvisited cells are appended in XZ-order to form a complete traversal, which is then used for reward estimation.

In a KV store, query efficiency is largely determined by interval localization and range scans after mapping the query window to one-dimensional keys. A desirable traversal should produce a small number of query sections and, when multiple sections remain, keep them locally coherent in the key space. Following prior work [15] on cohesion measures, we characterize this property using the distribution of gaps between consecutive query sections. If the generated query sections are $[l_1, r_1], [l_2, r_2], \ldots, [l_m, r_m]$ with $r_i < l_{i+1}$, the i-th gap is defined as $n_i = l_{i+1} - r_i$.

For example, consider two traversals that both produce the same three query sections. One yields intervals $[1, 3]$, $[6, 8]$, and $[11, 13]$, whose gap lengths are $(3, 3)$. Another yields $[1, 3]$, $[5, 7]$, and $[11, 13]$, whose gap lengths are $(2, 4)$. Although both have the same number of sections and the same total gap length, the former has higher gap entropy. This indicates more regular spacing between query sections, leading to more stable page transitions during scans and fewer irregular long jumps across key regions. Accordingly, we adopt the following cohesion objective:

$$\text{coh}(\mathcal{W}, \pi) = -n_{\text{rng}} \sum_i p_i \log p_i, \quad p_i = \frac{n_i}{\sum_j n_j}. \tag{16}$$

where n_{rng} is the number of scan ranges, and n_i is the length of the i-th gap between consecutive ranges. The factor n_{rng} captures fragmentation, while the entropy term captures the regularity of inter-range spacing. When the number of ranges is fixed, larger entropy corresponds to stronger local cohesion and more predictable scan behavior. The global reward is defined relative to the baseline XZ-order π_z:

$$\text{Rew}_{\text{global}} = \frac{\text{coh}(\mathcal{W}, \pi) - \text{coh}(\mathcal{W}, \pi_z)}{\text{coh}(\mathcal{W}, \pi_z)}, \tag{17}$$

Policy Learning. LETI learns the traversal policy using an Actor-Critic framework [16]. Given a state s_t, the Actor network outputs a probability distribution over candidate actions. To enforce the locality-constrained transition rule, a state-dependent mask is applied so that invalid cells, i.e., non-neighboring or previously visited nodes, receive zero probability before normalization. The Critic network estimates the state value v_t, which serves as a baseline for variance-reduced optimization. At each step, the agent samples an action from the masked policy, updates the partial traversal, and receives the corresponding reward.

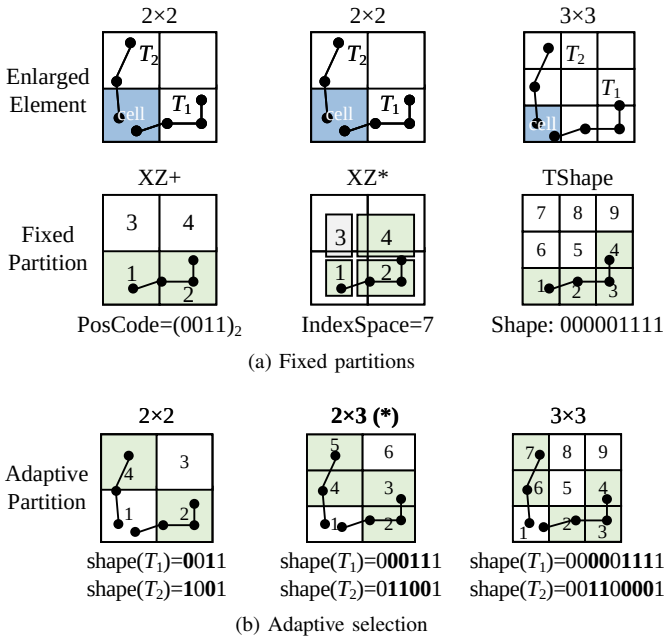
After a complete traversal is constructed, LETI computes advantage estimates using Generalized Advantage Estimation (GAE) [17], and updates both the Actor and Critic networks using Proximal Policy Optimization (PPO) [18]. This combination stabilizes training in the sequential combinatorial setting and produces a traversal policy that extends beyond fixed traversal orders.

C. Adaptive Shape Encoding

Quadtree-based spatial encoding captures only coarse trajectory location. To support finer pruning, prior work [7], [9], [10] augments each spatial encoding with a shape encoding over the corresponding enlarged element, as illustrated in Fig. 9.

Existing methods use fixed partitioning schemes. As shown in Fig. 9(a), XZ2+ [7] and XZ* [9] use a fixed 2×2 EE and a matching 2×2 partition. XZ2+ encodes coverage as a bitmap, while XZ* maps coverage patterns to compact integer identifiers. TShape [10] follows the same design principle, but supports larger EE and finer partitions. Although these designs differ in encoding format, they apply one partitioning rule to all trajectories within the same EE. This coupling limits shape filtering under heterogeneous trajectory distributions. Coarse partitions may provide insufficient separation, whereas finer partitions enlarge the shape domain exponentially (up to $2^{n_x \times n_y}$), increasing encoding space and index size without proportional pruning benefit.

Rather than fixing one partition per EE type, LETI selects a partition for each EE based on its assigned trajectories, aiming to use as few bits while preserving shape discriminability. Fig. 9(b) illustrates this trade-off. For the same EE, a 2×2 partition distinguishes trajectories T_1 and T_2 in only two bits.



(a) Fixed partitions

(b) Adaptive selection

Fig. 9. Adaptive shape encoding with fixed vs. adaptive partitions. (a) Existing methods use a fixed grid per EE. (b) LETI selects a compact partition based on local trajectory distribution, balancing encoding size and discriminability.

A 3×3 partition increases this to five bits, but requires a 9-bit encoding. A 2×3 partition yields four differing bits using only 6 bits, while avoiding weakly informative or empty subcells. It thus provides stronger discrimination than 2×2, while using fewer bits and higher spatial utilization than 3×3.

To quantify this trade-off, we evaluate each candidate partition using two metrics: (i) an *occupancy ratio* $\rho(c, n_x, n_y)$ that measures utilization of the $n_x \times n_y$ grid, and (ii) a *discriminability* score $\kappa(c, n_x, n_y)$ that measures separability of trajectory shapes. We combine them into a utility score

$$\text{score}(c, n_x, n_y) = \rho(c, n_x, n_y) \times \kappa(c, n_x, n_y). \tag{18}$$

A larger score indicates better partition utilization and stronger shape separation. The occupancy ratio is defined as

$$\rho(c, n_x, n_y) = \frac{|c_{\text{cross}}|}{n_x \times n_y}, \tag{19}$$

where $|c_{\text{cross}}|$ denotes the number of subcells intersected by at least one trajectory. The discriminability score is computed from the normalized Hamming distance [19] between shape encodings:

$$\bar{d}_{\text{ham}} = \frac{d_{\text{ham}}(s(\mathcal{T}_1), s(\mathcal{T}_2))}{n_x \times n_y}. \tag{20}$$

Let $M = \binom{|\mathcal{T}(c)|}{2}$ be the number of trajectory pairs in cell c. We define discriminability as the average pairwise distance:

$$\kappa(c, n_x, n_y) = \frac{1}{M} \sum \bar{d}_{\text{ham}}. \tag{21}$$

Given these metrics, LETI determines the partition via a grid search over candidate grid sizes. Specifically, we enumerate (n_x, n_y) within a bounded range and select the configuration that maximizes the utility score $\text{score}(c, n_x, n_y)$. In practice, we set $n_x, n_y \in [2, 8]$. This range ensures that the resulting bitmap length does not exceed $n_x \times n_y \leq 64$ bits (i.e., 8 bytes), while still providing sufficient flexibility to adapt to local trajectory distributions. Let P denote the number of candidate configurations in the search space. For each candidate, we compute the occupancy ratio and estimate the discriminability score using at most 100 sampled trajectory pairs. The overall time complexity is therefore $O(P)$.

V. Query Processing

LETI supports spatial range query (SRQ), similarity search, and top-k queries under a unified filter-and-refine framework. This section focuses on SRQ, while the same key organization can be reused for similarity-based queries by replacing geometric refinement with distance evaluation.

A. Auxiliary Structures

To support efficient query processing, LETI stores trajectories in the primary table and maintains two auxiliary structures for each retained cell $c \in \mathcal{C}$. The *shape cache* $\text{ShapeCache}(c)$ stores the distinct shape tuples $\langle s,\ s^* \rangle$, that occur in cell c, where s is a raw shape bitmap and s^* is its corresponding compact local shape code. These entries support shape filtering and key-prefix reconstruction within the cell.

We also maintain a contained-interval cache, denoted by $\text{ContainedCache}(c)$, which stores the key intervals of all retained descendant cells in the subtree rooted at c. Since trajectories in a cell c map to a contiguous key interval, each cell corresponds to the half-open interval

$$[\pi(c) \ll b,\ (\pi(c) + 1) \ll b). \tag{22}$$

$\text{ContainedCache}(c)$ thus stores the union of these intervals for all retained descendants, allowing LETI to directly retrieve all relevant trajectories when c is fully covered, without traversing the subtree.

B. Spatial Range Query Processing

Given a query window q, LETI evaluates an SRQ through three stages: cell localization, candidate scan range generation, and exact refinement.

(1) Cell Localization. LETI traverses the original quadtree in breadth-first order. Pruned cells are used only for routing, whereas retained cells in $\mathcal{C}$ participate in candidate generation. For each visited retained cell c, let $E_c = \text{EE}(c)$. Cells satisfying $E_c \cap q \neq \varnothing$ are identified as candidate cells for further processing.

(2) Candidate Scan Range Generation. For each candidate cell c, LETI distinguishes two cases. (1) If $E_c \subseteq q$, all trajectories in the subtree rooted at c are candidates. Instead of descending into the subtree, LETI directly returns the pre-materialized key intervals $\text{ContainedCache}(c)$ as scan ranges, which avoids further traversal overhead. (2) If $E_c \cap q \neq \varnothing$ but $E_c \nsubseteq q$, LETI performs shape-based filtering. It first computes a query signature $\sigma_q(c)$ over $\text{EE}(c)$ and matches it against entries in $\text{ShapeCache}(c)$. For each tuple $\langle s, s^* \rangle \in \text{ShapeCache}(c)$ satisfying $\sigma_q(c) \wedge s \neq 0$, LETI reconstructs the

Algorithm 2 Spatial Range Query Processing in LETI

Input: Query window q, root node r, retained cell set $\mathcal{C}$, maximum quadtree depth h
Output: Candidate scan ranges $\mathcal{R}$

1: $\mathcal{L}_{\text{bfs}} \leftarrow \text{children}(r)$
2: $\mathcal{R} \leftarrow \varnothing$
3: **while** $\mathcal{L}_{\text{bfs}} \neq \varnothing$ **do**
4: $\quad c \leftarrow \text{pop}(\mathcal{L}_{\text{bfs}})$
5: $\quad$ **if** $c \in \mathcal{C}$ **then**
6: $\quad\quad$ **if** $E_c \subseteq q$ **then**
7: $\quad\quad\quad \mathcal{R} \leftarrow \mathcal{R} \cup \text{ContainedCache}(c)$
8: $\quad\quad$ **else if** $E_c \cap q \neq \varnothing$ **then**
9: $\quad\quad\quad$ compute the query signature $\sigma_q(c)$
10: $\quad\quad\quad$ **for** each $\langle s, s^* \rangle \in \text{ShapeCache}(c)$ **do**
11: $\quad\quad\quad\quad$ **if** $\sigma_q(c) \wedge s \neq 0$ **then**
12: $\quad\quad\quad\quad\quad k_c \leftarrow (\pi(c) \ll b) \,|\, s^*$
13: $\quad\quad\quad\quad\quad \mathcal{R} \leftarrow \mathcal{R} \cup \{[k_c, k_c + 1)\}$
14: $\quad$ **if** $\text{level}(c) < h$ **then**
15: $\quad\quad \mathcal{L}_{\text{bfs}} \leftarrow \mathcal{L}_{\text{bfs}} \cup \text{children}(c)$
16: **return** $\mathcal{R}$

corresponding key prefix $k_c = ((\pi(c) \ll b) \,|\, s^*)$, and emits the singleton half-open interval $[k_c, k_c + 1)$. The generated singleton key intervals and cached intervals together form candidate scan ranges. Adjacent ranges can be merged before issuing KV scans.

Algorithm 2 summarizes the procedure. It takes a query window q, the quadtree root r, and retained cell set $\mathcal{C}$ as input, and outputs candidate scan ranges $\mathcal{R}$. The algorithm initializes a BFS queue from the root children (Line 3) and iteratively visits cells until the queue is exhausted (Lines 5–21). For each cell c, it checks whether c is retained (Line 7) and evaluates E_c (Line 8). Fully covered cells contribute precomputed ranges from $\text{ContainedCache}(c)$ (Lines 9–10), while intersecting cells trigger shape filtering via $\text{ShapeCache}(c)$ (Lines 11–16), which may emit singleton ranges from retained shape tuples (Line 15). Child nodes are then added to the queue (Lines 18–20), and the procedure returns the final scan ranges.

(3) KV Scan and Refinement. After candidate scan ranges are generated, LETI directly issues range scans to the KV store and retrieves candidate trajectories. Exact geometric intersection checks are then applied to remove false positives and produce final query results.

The cost of the above procedure is dominated by cell traversal and shape-based filtering. During traversal, each quadtree node is visited at most once in the BFS loop (Lines 5–21), yielding a cost of $O(N_q)$, where N_q is the number of visited nodes. Filtering depends on the number of shape-cache entries examined. In particular, $M_q = \sum_{c \in \mathcal{C}_q} |I(c)|$ counts the total number of accessed entries (Lines 13–16), contributing $O(M_q)$ time. Thus, candidate scan range generation takes $O(N_q + M_q)$ time. If emitted key prefixes are ordered, range merging is linear in the number of intervals and does not affect the overall complexity. KV scans and exact refinement are applied only to the returned candidates.

TABLE II
EXPERIMENTAL SETTINGS.

Parameter	Values
Maximum quadtree depth h	6, 7, **8**, 9, 10
Pruning threshold δ	2, **4**, 6, 8, 10
Workload distribution $\mathcal{D}$	Uniform, **Skewed**, Gaussian
Query window size r	100, 500, **1000**, 1500, 2000

VI. EXPERIMENTS

A. Experimental Settings

Datasets. We evaluate LETI on two real-world trajectory datasets and their scaled variants. All trajectories are pre-segmented before indexing following prior studies [20]–[22].

- **CD-Taxi** [23]. Contains 48,223 trajectories from 2,940 taxis in Chengdu, China, collected over 10 days (Oct 1–10, 2018) with an average sampling interval of 3.13s, covering (30.65–30.73°N, 104.04–104.13°E).
- **T-Drive** [24]. Contains GPS records from 10,357 taxis in Beijing (Feb 2–8, 2008), with about 15M points and an average sampling interval of 177s. We extract 318,774 trajectories for indexing, covering (39.6–40.1°N, 116.0–116.8°E).
- **T-Drive-Large**. Constructed for scalability evaluation by generating $\times 100$, $\times 500$, $\times 1000$, and $\times 2000$ variants from the original 530 MB dataset (up to 1.04 TB). To avoid duplication, samples are synthesized via data augmentation, including random perturbation, shape-preserving Gaussian noise, interpolation, random rotation/scaling, and feature-level mixing.

Query Workloads. We evaluate LETI using spatial range queries (SRQ) as the primary workload, along with similarity search and top-k queries for completeness.

For SRQ, we generate query workloads under three spatial distributions: *uniform*, *skewed*, and *Gaussian*. For each distribution, 100 queries are generated for each of five window sizes, ranging from $100 \times 100\,\text{m}^2$ to $2000 \times 2000\,\text{m}^2$.

Under the *uniform* setting, query centers are uniformly sampled from trajectory points and expanded into rectangular windows. For non-uniform workloads, *skewed* queries sample 80% of centers from hotspot regions and 20% from all points, where hotspots correspond to the top 20% grid cells with the highest trajectory density, while *Gaussian* queries are drawn from a distribution fitted to trajectory points, with samples clipped to the spatial bounding box.

Similarity and top-k queries are constructed following the *skewed* distribution by sampling 100 query trajectories from the T-Drive dataset. Their evaluation follows the standard filter-and-refine process in [9].

Baselines. We compare LETI with representative trajectory encodings based on XZ-order and learned SFC methods.

- **XZ*** [9]. Extends XZ-order by associating each trajectory with an EE partitioned into a fixed 2×2 grid, where the occupancy pattern serves as a coarse shape descriptor.
- **TShape** [10]. Generalizes XZ* by using a larger EE and finer partitioning, and maps shape bitmaps to compact

TABLE III
QUERY LATENCY (MS) UNDER DIFFERENT WORKLOADS AND QUERY RANGES.

Dataset	Method	Skewed					Gaussian					Uniform				
		100	500	1000	1500	2000	100	500	1000	1500	2000	100	500	1000	1500	2000
T-Drive	LETI	**17**	**18.1**	**20.3**	**24.8**	**27.6**	**9.1**	**10.4**	**12.7**	**14.6**	**16.2**	**6.1**	**6.8**	**7.6**	**8.9**	**10.2**
	XZ*	26.6	28.1	30.7	35.2	38.6	16.8	18.2	20.1	22.8	25.1	14.6	15.8	17.2	18.9	20.3
	TShape	24.3	26.2	29.4	33.8	37.1	11.4	12.6	14.2	16.5	18.3	6.9	7.5	8.6	9.8	11.4
	LMSFC	117.5	119.2	122.8	126.5	130.1	100.6	103.2	106.8	110.5	114.2	90.3	94.7	98.6	102.3	106.1
	BMTree	52.4	55.8	60.2	65.7	70.1	41.9	45.3	48.7	52.8	56.4	42.7	46.1	49.8	53.6	57.2
CD-Taxi	LETI	**15.3**	**16.3**	**19.4**	**22.3**	**24.8**	**8.2**	**9.4**	**11.4**	**13.1**	**14.6**	**5.5**	**6.1**	**6.8**	**8**	**9.2**
	XZ*	25	26.4	28.9	33.1	36.3	15.8	17.1	18.9	21.4	23.6	13.7	14.9	16.2	17.8	19.1
	TShape	22.4	24.1	27	31.1	34.1	10.5	11.6	13.1	15.2	16.8	6.3	6.9	7.9	9	10.5
	LMSFC	112.8	114.4	117.9	121.4	124.9	96.6	99.1	102.5	106.1	109.6	86.7	90.9	94.7	98.2	101.9
	BMTree	48.7	51.9	56	61.1	65.2	39	42.1	45.3	49.1	52.5	39.7	42.9	46.3	49.8	53.2

integer codes to improve filtering granularity (we use a 3×3 setting in our implementation).

- **LMSFC** [25]. Learns a data-aware SFC via Sequential Model-Based Optimization (SMBO), aiming to improve spatial locality for range queries.
- **BMTree** [26]. Learns a data-aware SFC traversal using Monte Carlo Tree Search (MCTS), with the objective of optimizing spatial locality under given workloads.

Metrics and Parameter Settings. We use average query latency as the primary end-to-end metric. To interpret performance differences, we also report the number of retained cells (RC), row-key intervals (RKI), and candidate trajectories (CT). Training time is included in parameter studies to measure the cost of learning traversal orders.

We evaluate the impact of both index and workload parameters. The quadtree depth h controls spatial granularity, and the pruning threshold δ determines whether a cell is retained. On the workload side, the query distribution $\mathcal{D}$ defines the spatial access pattern, while the window size r controls the query range. Table II summarizes all evaluated settings, with default values highlighted in bold.

Implementation Details. All methods are implemented within the TMan [10] system to ensure a fair and unified comparison. Since LMSFC and BMTree are designed for point data, they are adapted to trajectories via SFC monotonicity [25]. Each trajectory is mapped to a 1D interval using the SFC codes of the lower-left and upper-right corners of its MBR, enabling interval-based filtering. The same geometric refinement procedure is applied across all methods.

For LETI, LMSFC, and BMTree, a traversal order is trained for each combination of workload distribution $\mathcal{D}$, depth h, and pruning threshold δ using the full dataset, and then applied during query processing.

All experiments are conducted on a single machine unless otherwise specified. Distributed experiments run on an eight-node cluster using HBase [27] as the KV store. Each node is equipped with an Intel Xeon E-2224G CPU (4 cores, 3.50 GHz), 128 GB RAM, and two 3.6 TB local disks.

B. Query Performance

We evaluate average query latency of LETI and all baselines under varying query distributions and window sizes on both datasets. Results in Table III show that LETI consistently achieves the best performance.

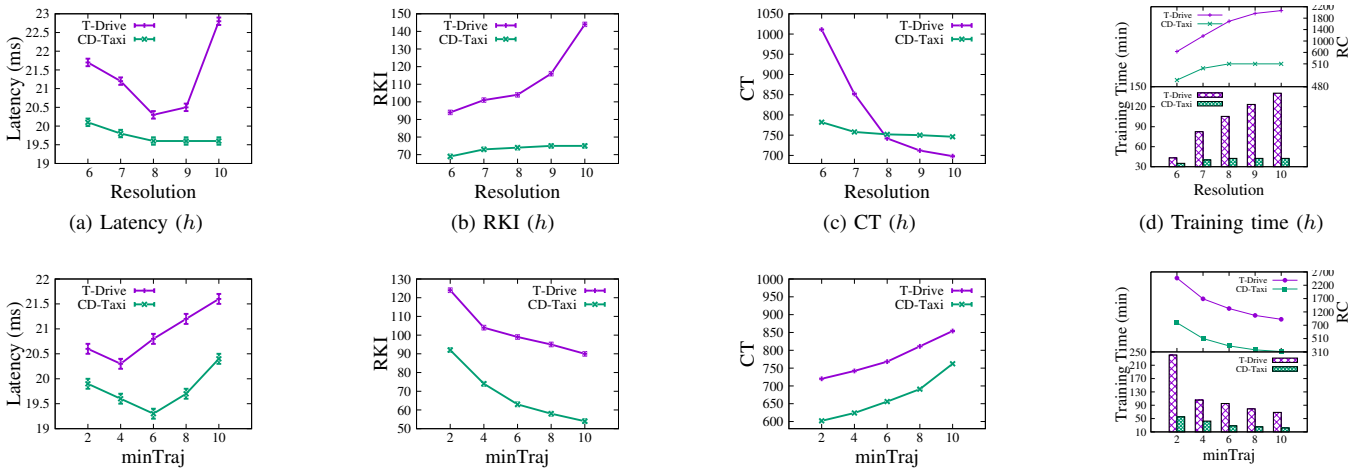
Among baselines, TShape performs the strongest overall. Nevertheless, LETI still reduces latency by 9.2%–30.9% on T-Drive and 11.1%–32.4% on CD-Taxi. The largest gains occur under skewed distributions (e.g., 20.3 ms vs. 29.4 ms on T-Drive and 16.3 ms vs. 24.1 ms on CD-Taxi), as the learned SFC better aligns frequently co-accessed cells in key space, improving scan locality over XZ-order. Under uniform queries, LETI still achieves around 10% improvement over TShape, indicating that optimizing traversal using data distribution remains beneficial. It places cells that often co-occur closer in key space.

For all methods, latency increases with larger query windows, as more cells intersect the query region, raising filtering and refinement costs. However, LETI is less sensitive to window size than baselines. On both datasets, LETI and TShape grow more slowly (e.g., from 7.1 ms to 8.8 ms on CD-Taxi), due to finer spatial partitioning and stronger shape filtering. LETI further limits candidates entering refinement as regions grow. In contrast, learned SFC methods such as LMSFC and BMTree consistently underperform XZ-based approaches, as their keys lack shape information. Filtering thus relies only on interval intersection, leading to more false positives and higher refinement cost.

C. Parameter Sensitivity

LETI builds trajectory encodings on a compact quadtree, controlled by two parameters: maximum depth h and pruning threshold δ. The former determines spatial granularity, while the latter controls removal of low-density cells. We analyze their impact on query latency using RKI and CT, and report training time to quantify learning cost.

Effect of Depth h. Fig. 10(a)–(d) show that increasing h introduces a trade-off between filtering precision and scan fragmentation. On T-Drive, latency drops from 21.7 ms at $h = 6$ to 20.3 ms at $h = 8$, then rises to 22.8 ms at $h = 10$. Deeper partitioning affects CT and RKI in opposite ways. As h increases, finer granularity strengthens filtering, reducing CT from 1011 to 698 (Fig. 10(c)). Meanwhile, retained cells grow from 631 to 2065, increasing fragmentation and RKI from 94 to 144 (Fig. 10(b)). Moderate depth improves filtering enough to offset scan cost, while excessive depth reverses this balance.



(a) Latency (h) (b) RKI (h) (c) CT (h) (d) Training time (h)

(e) Latency (δ) (f) RKI (δ) (g) CT (δ) (h) Training time (δ)

Fig. 10. Parameter sensitivity of LETI under varying quadtree depth h and pruning threshold δ.

This effect is weaker on CD-Taxi. As h increases, RC changes slightly (489 to 511), RKI remains nearly stable (71 to 74), and CT declines from 782 to 520. With $\delta = 4$, most new cells are pruned early, so the structure stabilizes quickly. Given the more concentrated distribution, deeper partitioning mainly improves local filtering without increasing fragmentation, leading to only minor latency changes (20.1 ms to 19.5 ms).

Training time follows a similar pattern. On T-Drive, increasing h enlarges the retained search space, raising training time from 43.2 min to 140.3 min. On CD-Taxi, the increase is modest (35.1 min to 42.5 min), again due to early saturation of retained cells.

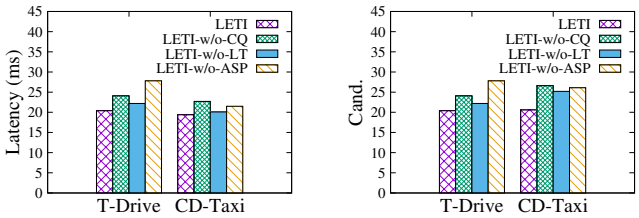
Effect of Threshold δ. Fig. 10(e)–(h) show that δ balances structural compactness and filtering fidelity. On T-Drive, average latency is minimized at $\delta = 4$. Increasing δ from 2 to 10 reduces RC from 2470 to 924 and RKI from 124 to 90, indicating a more compact structure with less fragmented scans. Meanwhile, spatial encoding becomes coarser: more trajectories are absorbed into higher-level cells, and CT rises from 720 to 854. Latency thus reflects a trade-off: smaller δ preserves filtering but incurs higher scan cost, whereas larger δ reduces scan cost at the expense of weaker filtering.

CD-Taxi shows a different trend. As δ increases, RC decreases from 795 to 315, RKI from 92 to 54, and CT from 602 to 438. Stronger pruning improves both scan compactness and filtering, leading to a monotonic latency drop from 19.9 ms to 19.0 ms. This suggests that many low-density cells contribute little to discrimination and mainly cause fragmentation; pruning simplifies the structure without degrading representation.

Training time decreases monotonically with δ on both datasets, as stronger pruning reduces retained cells and shrinks the search space. The effect is more pronounced on T-Drive, where training time drops from 240.7 min to 68.6 min.

D. Ablation Study

We compare full LETI with three variants: *LETI-w/o-CQ* removes the compact quadtree and learns traversal over the original spatial order, *LETI-w/o-LT* keeps the compact quadtree but uses XZ-order, and *LETI-w/o-ASP* replaces adaptive shape partitioning with a fixed one. Fig. 11a reports average latency under the skewed workload with $r = 1000$ m.



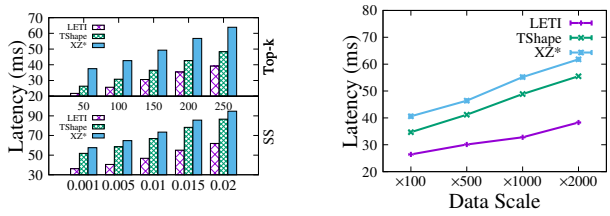
(a) Latency. (b) Candidates.

Fig. 11. Ablation study of LETI.

All components contribute to performance. Within spatial encoding, removing the compact quadtree causes the largest degradation: latency rises from 20.4 ms to 24.1 ms on T-Drive and from 19.4 ms to 22.7 ms on CD-Taxi. Thus, the gain of LETI is not due to learned traversal alone. The compact quadtree prunes sparse cells and focuses learning on relevant regions; without it, locality weakens and scans become fragmented.

Replacing the learned traversal with XZ-order introduces smaller but consistent overhead, increasing latency to 22.2 ms on T-Drive and 20.1 ms on CD-Taxi (+8.8% and +3.6%). Even with the same retained cells, ordering still affects scan efficiency. The learned traversal places frequently co-accessed cells closer in key space, reducing interval fragmentation.

The impact of ASP is dataset-dependent. On T-Drive, removing ASP causes the largest increase (27.8 ms, +36.3%), while on CD-Taxi the increase is smaller (21.5 ms, +10.8%). Fig. 11(b) shows that fixed partitioning introduces more false positives, raising refinement cost. The stronger effect on T-Drive suggests that its broader spatial coverage and more diverse local patterns benefit more from adaptive partitioning, whereas on CD-Taxi the compact quadtree remains dominant.



(a) Similarity and top-k. (b) Scalability.

Fig. 12. Additional query types and distributed scalability of LETI.

E. Additional Query Types

We further evaluate LETI on similarity search and top-k queries over the skewed T-Drive workload. For similarity search, the Fréchet threshold ranges from 0.001 to 0.02, and for top-k we vary k from 50 to 250.

As illustrated in Fig. 12(a), LETI consistently outperforms both baselines. For similarity search, latency increases from 36.2 ms to 61.9 ms as the threshold grows, while TShape rises from 51.8 ms to 86.7 ms and XZ* from 57.6 ms to 94.8 ms, showing effective pruning even as candidate sets grow.

A similar trend appears for top-k queries. As k increases from 50 to 250, LETI grows from 21.8 ms to 39.2 ms, compared with 26.4–48.3 ms for TShape and 37.5–63.9 ms for XZ*. The gap widens at larger k, where tighter scan ranges and more selective shape filtering eliminate more candidates before ranking. These results show that LETI remains effective beyond spatial range queries, including distance- and ranking-based workloads.

F. Distributed Scalability

We evaluate LETI under increasing data sizes. Following Section VI-A, we construct four enlarged T-Drive variants and run all experiments on the same eight-node cluster, where $\times N$ denotes scaling by N. We compare LETI with TShape and XZ*. Fig. 12(b) reports average latency.

LETI consistently achieves the lowest latency. As the data size grows from $\times 100$ to $\times 2000$, latency increases from 26.4 ms to 38.3 ms for LETI, compared with 34.7–55.5 ms for TShape and 40.6–61.8 ms for XZ*. LETI grows more slowly, and the advantage becomes more pronounced at larger scales.

This stems from jointly reducing scan overhead and refinement cost. The compact quadtree limits accessed cells, the learned traversal keeps co-accessed regions close in key space, and adaptive partitioning filters more false positives before refinement. As data volume grows, these effects accumulate and slow latency growth relative to baselines. In particular, the gap widens from 8.3 ms at $\times 100$ to 17.2 ms at $\times 2000$ against TShape, showing that LETI maintains its advantage even though scale increases.

VII. Related Work

A. Trajectory Data Management Systems

Trajectory data management has been studied in both distributed and database systems. Systems such as HadoopTrajectory [28] and TrajSpark [29] leverage Hadoop and Spark for scalable trajectory analytics, while Distributed MobilityDB [30] extends MobilityDB with distributed query capabilities for spatiotemporal data.

To accelerate query processing, prior work explores specialized indexing and query optimization techniques. BT-Tree [31] organizes trajectories using hierarchical spatial partitions, while Dragoon [32] improves efficiency through index-assisted pruning. These approaches typically follow a filter-and-refine paradigm, where spatial approximations (e.g., MBRs) generate candidates before exact verification.

A separate line of work studies trajectory indexing in ordered KV stores. TrajMesa [7] adopts XZ-order-based encodings to map spatial objects into one-dimensional keys for efficient range scans, while UlTraMan [33] supports trajectory queries via space linearization in KV storage.

B. SFC-based Spatial and Trajectory Indexing

Space-filling curves (SFCs) are widely used to linearize multi-dimensional data into one-dimensional keys for indexing in ordered KV stores [34], supporting spatial queries via range scans over the key space.

Point-based indexing. Lots of work applies SFCs to point data, mapping coordinates directly to one-dimensional keys. ZM-Index [35] and HMIndex [36] combine Z-order or Hilbert curves with learned models to predict data positions. Instance-Optimal Z-Index [37] refines partitioning and traversal to reduce false positives, while WAZI [38] groups frequently co-accessed points to improve locality. These methods are not directly applicable to trajectories. A common adaptation is to decompose trajectories into points, which introduces redundancy and limits pruning.

Trajectory indexing. Prior work on trajectories adopts object-level encodings, typically based on MBRs combined with XZ-order [39]. GeoMesa [6] implements the XZ2 index, and subsequent methods improve intra-region representation. XZ2+ [7] augments MBR encoding with positional signatures, XZ* [9] refines spatial encoding via region combinations, and TShape [10] incorporates multi-scale shape partitions. These methods reduce redundancy and improve filtering, but their spatial organization is largely determined by predefined partitions, limiting adaptability under skewed data and workloads.

C. Learning-based SFC Optimization

To improve adaptability, recent work explores optimizing SFC mappings based on data distribution and query workload.

Analytical approaches. Prior studies [40], [41] analyze the clustering properties of SFCs and their impact on range query performance. The Onion Curve [42] achieves near-optimal clustering for regular query shapes. While these results provide useful insights, they rely on simplified assumptions and are less effective under heterogeneous workloads.

Cost-based selection. Another line of work formulates SFC optimization as selecting a mapping from a predefined candidate set. QUILTS [15] enumerates SFCs using bit merging patterns and selects the one minimizing estimated query cost, while subsequent work [43] improves estimation efficiency.

These methods incorporate workload information explicitly, but still rely on a single global mapping across the space.

Learning-based construction. More recent approaches construct SFCs using learning techniques. LMSFC [25] models SFCs as parameterized mappings optimized via SMBO, and BMTree [26] applies reinforcement learning to generate piecewise mappings. These approaches improve flexibility, but primarily focus on optimizing the linearization function itself, without considering trajectory-level encoding or hierarchical structure.

Existing indexing methods either rely on fixed spatial encodings or optimize SFC mappings in isolation. LETI addresses this limitation by integrating compact hierarchy construction, learned traversal, and adaptive shape encoding in a unified framework.

VIII. Conclusion

This paper proposes LETI, an efficient trajectory indexing framework for KV stores. Extensive experiments on real-world datasets verify the strong performance of LETI. In particular, LETI reduces query latency by 9.2%–30.9% on T-Drive and 11.1%–32.4% on CD-Taxi over the strongest baseline. Moreover, LETI consistently outperforms existing methods on similarity search, top-k queries, and large-scale distributed settings. In future work, we plan to extend LETI to online workload adaptation and spatio-temporal joint indexing.

References

[1] Y. Zheng, "Trajectory data mining: An overview," *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 6, no. 3, pp. 1–41, 2015.

[2] S. Ding, Z. Li, K. Zhang, and F. Mao, "A comparative study of frequent pattern mining with trajectory data," *Sensors*, vol. 22, no. 19, p. 7608, 2022.

[3] J. Jiang, K. Yan, X. Xia, and B. Yang, "A survey of deep learning-based pedestrian trajectory prediction: Challenges and solutions," *Sensors*, vol. 25, no. 3, p. 957, 2025.

[4] J. Tian, Z. Zhao, and Z. Ding, "Next point-of-interest recommendation based on joint mining of spatial–temporal and semantic sequential patterns," *ISPRS International Journal of Geo-Information*, vol. 12, no. 7, p. 297, 2023.

[5] R. Li, X. He, Y. Sun, J. Jiang, Y. Shang, G. Li, and C. Chen, "Spatio-temporal keyword query processing based on key-value stores," *Data Science and Engineering*, vol. 10, no. 1, pp. 98–116, 2025.

[6] J. N. Hughes, A. Annex, C. N. Eichelberger, A. Fox, A. Hulbert, and M. Ronquest, "Geomesa: A distributed architecture for spatio-temporal fusion," in *Proceedings of the SPIE Conference on Geospatial Informatics, Fusion, and Motion Video Analytics V*, vol. 9473. Baltimore, MD, USA: SPIE, 2015, pp. 128–140.

[7] R. L. et al., "Trajmesa: Distributed storage engine for trajectory data," in *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. Dallas, TX, USA: IEEE, 2020, pp. 2002–2005.

[8] ——, "Trajmesa: A distributed nosql-based trajectory data management system," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 1, pp. 1013–1027, 2023.

[9] H. He, R. Li, S. Ruan, T. He, J. Bao, T. Li, and Y. Zheng, "Trass: Efficient trajectory similarity search based on key-value data stores," in *Proc. IEEE Int. Conf. Data Eng. (ICDE)*. Kuala Lumpur, Malaysia: IEEE, 2022, pp. 2306–2318.

[10] H. He, Z. Xu, R. Li, J. Bao, T. Li, and Y. Zheng, "Tman: A high-performance trajectory data management system based on key-value stores," in *Proc. IEEE Int. Conf. Data Eng. (ICDE)*. Utrecht, Netherlands: IEEE, 2024, pp. 4951–4964.

[11] R. A. Finkel and J. L. Bentley, "Quad trees: A data structure for retrieval on composite keys," *Acta Informatica*, vol. 4, no. 1, pp. 1–9, 1974.

[12] H. Alt and M. Godau, "Computing the fréchet distance between two polygonal curves," *International Journal of Computational Geometry & Applications*, vol. 5, no. 1–2, pp. 75–91, 1995.

[13] H. Sakoe and S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 26, no. 1, pp. 43–49, 1978.

[14] P.-N. Tan, M. Steinbach, and V. Kumar, *Introduction to Data Mining*. Addison-Wesley, 2005.

[15] S. Nishimura and H. Yokota, "Quilts: Query-aware space-filling curves," in *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*. Chicago, IL, USA: ACM, 2017, pp. 1525–1537.

[16] V. R. Konda and J. N. Tsitsiklis, "Actor-critic algorithms," in *Advances in Neural Information Processing Systems (NeurIPS 12)*. Cambridge, MA, USA: MIT Press, 1999, pp. 1008–1014.

[17] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proceedings of the International Conference on Learning Representations (ICLR)*. San Juan, Puerto Rico: ICLR, 2016, pp. 1–14.

[18] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017.

[19] R. W. Hamming, "Error detecting and error correcting codes," *Bell System Technical Journal*, vol. 29, no. 2, pp. 147–160, 1950.

[20] R. Li, S. Ruan, J. Bao, Y. Li, Y. Wu, and Y. Zheng, "Querying massive trajectories by path on the cloud," in *Proceedings of the 25th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, ser. SIGSPATIAL '17. Redondo Beach, California, USA: ACM, 2017, pp. 77:1–77:4.

[21] R. Li, S. Ruan, J. Bao, and Y. Zheng, "A cloud-based trajectory data management system," in *Proceedings of the 25th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, ser. SIGSPATIAL '17. Redondo Beach, California, USA: ACM, 2017, pp. 94:1–94:4.

[22] Y. Sun, F. Meng, R. Li, Y. Tang, C. Chen, and J. Zhong, "Streaming trajectory segmentation based on stay-point detection," in *International Conference on Database Systems for Advanced Applications (DASFAA)*, ser. Lecture Notes in Computer Science. Cham: Springer, 2024, pp. 203–213.

[23] Y. Sun, T. Huang, Z. Zhang, R. Li, C. Chen, and Y. Zheng, "Ear-mm: An efficient adaptive and robust algorithm for streaming map matching," *IEEE Trans. Intell. Transp. Syst.*, vol. 27, no. 1, pp. 681–691, 2026.

[24] J. Yuan, Y. Zheng, X. Xie, and G. Sun, "T-Drive: Driving directions based on taxi trajectories," in *Proceedings of the 18th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (SIGSPATIAL GIS)*. San Jose, CA, USA: ACM, 2010, pp. 99–108.

[25] J. G. et al., "Lmsfc: Learned monotonic space filling curves," *PVLDB*, vol. 16, no. 10, pp. 2605–2617, 2023.

[26] J. L. et al., "Bmtree: Designing and learning piecewise space-filling curves," 2025.

[27] Apache Software Foundation, "Apache HBase," https://hbase.apache.org/, 2024.

[28] Z. He *et al.*, "Hadooptrajectory: A hadoop-based trajectory data processing system," in *IEEE BigData*, 2014.

[29] C. Zhang *et al.*, "Trajspark: A distributed in-memory trajectory data processing framework," in *IEEE ICDE*, 2017.

[30] E. Zimányi *et al.*, "Distributed mobilitydb: A scalable platform for moving object data," *VLDB Journal*, 2023.

[31] X. Li *et al.*, "Bt-tree: A high-performance index structure for trajectory data," in *SIGMOD*, 2023.

[32] J. Kim *et al.*, "Dragoon: Efficient trajectory query processing with index optimization," in *VLDB*, 2019.

[33] Y. Wang *et al.*, "Ultraman: Efficient trajectory management on kv stores," in *ICDE*, 2021.

[34] J. K. Lawder and P. J. H. King, "Using space-filling curves for multi-dimensional indexing," in *Advances in Databases (BNCOD)*. Berlin, Germany: Springer, 2000, pp. 20–35.

[35] H. W. et al., "Learned index for spatial queries," in *Proceedings of the IEEE International Conference on Mobile Data Management (MDM)*. Hong Kong, China: IEEE, 2019, pp. 569–574.

[36] N. Wang and J. Xu, "Spatial queries based on learned index," in *Spatial Data and Intelligence*. Cham, Switzerland: Springer, 2021, pp. 245–257.

[37] S. G. Pai, M. Mathioudakis, and Y. Wang, "Towards an instance-optimal z-index," in *Proceedings of the 4th International Workshop on Applied AI for Database Systems and Applications (AIDB@VLDB 2022)*. Sydney, Australia: VLDB Endowment, 2022, pp. 1–10.

[38] S. P. et al., "Wazi: A learned and workload-aware z-index," in *Proceedings of the International Conference on Extending Database Technology (EDBT)*. Paestum, Italy: VLDB Endowment, 2024, pp. 559–571.

[39] C. Böhm, G. Klump, and H.-P. Kriegel, "Xz-ordering: A space-filling curve for objects with spatial extension," in *Advances in Spatial Databases*, ser. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 1999, vol. 1651, pp. 75–90.

[40] B. M. et al., "Clustering properties of hilbert curve," *TKDE*, vol. 13, no. 1, pp. 124–141, 2001.

[41] P. Xu and S. Tirthapura, "Optimality of clustering properties of space-filling curves," *ACM Transactions on Database Systems (TODS)*, vol. 39, no. 2, pp. 1–27, 2014.

[42] P. Xu, C. Nguyen, and S. Tirthapura, "Onion curve: A space filling curve with near-optimal clustering," in *Proceedings of the IEEE 34th International Conference on Data Engineering (ICDE)*. Paris, France: IEEE, 2018, pp. 1236–1239.

[43] G. L. et al., "Efficient cost modeling of space-filling curves," *PVLDB*, vol. 17, no. 13, pp. 4773–4785, 2024.